Informe Técnico: Estándares de Codificación del Proyecto MONK GRAIN

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Un estándar de codificación es el acuerdo sobre cómo se escribe el código de un proyecto: cómo se nombran las variables, cómo se declaran las clases y los métodos, cómo se ordenan los archivos y qué se comenta. Su valor no es estético. Un proyecto que se escribe de una sola manera se lee más rápido, se corrige con menos riesgo y permite que alguien que llega después entienda la intención sin preguntar.

Este documento recoge los estándares que rigen **MONK GRAIN**, la plataforma de compra y venta de café pergamino seco. No son una propuesta: son las reglas que el proyecto aplica hoy sobre 99 archivos de Python y 83 de TypeScript, y que verifican automáticamente seis herramientas en cada envío al repositorio.

2. Objetivo

Definir el estándar de codificación de la plataforma según las tecnologías elegidas, especificando el nombramiento de variables, la declaración de clases, la declaración de métodos, la organización de los archivos y las reglas de comentario, y dejando constancia de cómo se hace cumplir cada regla.

3. Plataforma y estándares base adoptados

El proyecto combina dos lenguajes, y cada uno adopta el estándar de su comunidad en lugar de inventar uno propio. Escribir Python como si fuera Java —o al revés— produce código que nadie del ecosistema reconoce:

Capa	Tecnología	Estándar base
Backend	Python 3.12, FastAPI, SQLAlchemy 2	**PEP 8** y **PEP 257** (docstrings)
Frontend web	TypeScript, React 19, Next.js 15	Convenciones de la comunidad de React y TypeScript
Base de datos	PostgreSQL 16	`snake_case`, tablas en plural

4. Nombramiento de variables y elementos

La tabla de convenciones que rige todo el repositorio:

Elemento	Convención	Ejemplo del proyecto
Archivos y módulos Python	`snake_case`	`catalog_service.py`
Clases Python	`PascalCase`	`SettlementService`
Funciones y variables Python	`snake_case`	`create_sub_orders()`
Constantes Python	`SCREAMING_SNAKE_CASE`	`SETTLEMENT_HOLD_BUSINESS_DAYS`
Métodos internos	`snake_case` con guion bajo inicial	`_reserve_stock()`
Tablas y columnas	`snake_case`, tabla en plural	`sub_orders`, `producer_id`
Componentes React	`PascalCase`	`ProductCard.tsx`
Hooks de React	`camelCase` con prefijo `use`	`useCart.ts`
Rutas de la API	plural, en minúscula	`/api/v1/products`
Variables de entorno	`SCREAMING_SNAKE_CASE`	`DATABASE_URL`

4.1 Idioma de los identificadores

El código se escribe en **inglés** y se comenta en **español**. La razón es práctica: los marcos de trabajo, las librerías y el propio lenguaje están en inglés, y mezclar idiomas dentro de una misma expresión produce híbridos difíciles de leer como `getProductosCafe()`. Las explicaciones, en cambio, se escriben en el idioma en que se piensa el problema.

> **Excepción deliberada.** Los mensajes dirigidos al usuario van en español, porque los lee una persona en Colombia: «Solo quedan 40 kg disponibles.»

5. Declaración de clases

Una clase se declara con su nombre en `PascalCase` y una docstring inmediatamente debajo que explica qué representa y, cuando la decisión no es obvia, por qué está hecha así. El ejemplo es un modelo real del proyecto:

```
class Commission(Base, UUIDPrimaryKeyMixin):
    """Comisión de la plataforma sobre una sub-orden.

    `rate_applied` guarda la tasa vigente en el momento de confirmar: cambiar la
    comisión no altera pedidos ya calculados (HU-ADM-05, CA2).
    """

    __tablename__ = "commissions"

    sub_order_id: Mapped[uuid.UUID] = mapped_column(
        PgUUID(as_uuid=True),
        ForeignKey("sub_orders.id", ondelete="CASCADE"),
        unique=True,
        nullable=False,
    )
    rate_applied: Mapped[Decimal] = mapped_column(Numeric(5, 4), nullable=False)
    base_amount: Mapped[Decimal] = mapped_column(Numeric(14, 2), nullable=False)
```

Reglas que ilustra este fragmento:

- Nombre de clase en singular: una instancia es una comisión; la tabla, en plural.
- Docstring obligatoria, con la trazabilidad a la historia de usuario entre paréntesis.
- Atributos con anotación de tipo explícita (`Mapped[Decimal]`), no inferidos.
- Los importes monetarios usan `Decimal` y `Numeric`, nunca punto flotante.

6. Declaración de métodos

Un método declara el tipo de cada parámetro y el tipo de retorno. No es decoración: `mypy` rechaza el envío si falta una anotación o si un tipo no cuadra.

```
def _reserve_stock(self, variant_id: uuid.UUID, quantity: int) -> None:
    """Reserva kilos bloqueando la fila de inventario.

    `FOR UPDATE` serializa las compras concurrentes de la misma variante: sin
    él, dos checkouts simultáneos podrían leer el mismo stock y venderlo dos
    veces (HU-PRO-05, CA4).

    La reserva no descuenta el stock: lo retiene hasta que el pago se
    confirme. Un pago rechazado lo devuelve al catálogo.
    """
    inventory = self.session.scalar(
        select(Inventory).where(Inventory.variant_id == variant_id).with_for_update()
    )
    if inventory is None:
        raise NotFoundError("Esta presentación ya no está disponible.")

    if inventory.available_qty < quantity:
        raise InsufficientStockError(
            f"Solo quedan {inventory.available_qty} kg disponibles.",
            details={"variant_id": str(variant_id), "available": inventory.available_qty},
        )

    inventory.reserved_qty += quantity
    self.session.flush()
```

Regla	Cómo se ve en el ejemplo
Nombre en `snake_case`, verbo primero	`_reserve_stock` — dice qué hace
Guion bajo inicial si es interno	`_reserve_stock` no es parte de la interfaz pública
Tipos en parámetros y retorno	`variant_id: uuid.UUID`, `-> None`
Errores con excepción propia, no código de retorno	`NotFoundError`, `InsufficientStockError`
Mensaje de error en español y accionable	Dice cuántos kilos quedan, no solo que falló

7. Organización de archivos y regla de dependencias

El backend se organiza por **módulo de dominio**, no por tipo de archivo. Cada módulo agrupa lo suyo, y entre capas rige una dirección única:

> `router → service → repository → models`

- El **router** no consulta la base de datos ni contiene reglas de negocio: valida la entrada, resuelve la autorización y delega.

- El **service** es el único lugar donde se abren y confirman transacciones.
- El **repository** no conoce HTTP ni esquemas de entrada; solo entidades y consultas.
- Los **models** no importan nada de `modules/`.

Un módulo puede llamar al service de otro módulo, **nunca** a su repository. Esa restricción es la que permite extraer un módulo a un servicio independiente sin desenredar consultas cruzadas.

El frontend refleja esa organización: `web/src/features/` tiene una carpeta por dominio, en espejo con los módulos del backend.

8. Comentarios

La regla del proyecto es que **un comentario explica el porqué, no el qué**. El código ya dice qué hace; lo que no puede decir es qué alternativa se descartó y por qué razón. Un comentario que repite el código envejece mal y estorba.

Sobra	Sirve
`# Incrementa la cantidad reservada` `inventory.reserved_qty += quantity`	«La reserva no descuenta el stock: lo retiene hasta que el pago se confirme.»

Además, tres reglas de comentario obligatorias:

1. **Toda clase, módulo y método público lleva docstring** (PEP 257).
2. **Trazabilidad a la historia de usuario** entre paréntesis cuando la regla viene de un criterio de aceptación: `(HU-PRO-05, CA4)`.
3. **Una supresión de regla se justifica en la misma línea**: `# noqa: S105 — credencial de pruebas`.

9. Mensajes de versión

El historial también se escribe, y también tiene estándar. Una primera línea corta que dice qué cambió en lenguaje llano, y un cuerpo que explica por qué. Ejemplo real del repositorio:

> El porcentaje de la comisión, con su espacio
>
> `Intl` con `style: "percent"` devuelve «2,5%» pegado en es-CO, y en español el signo va separado del número. El resto del proyecto ya lo escribe así —«8 %», «87 % de cobertura»—, de modo que al centralizar la cifra en una constante se había colado una incoherencia tipográfica en los dos sitios que la muestran.

10. Cómo se hace cumplir el estándar

Un estándar que solo vive en un documento se incumple sin que nadie lo note. Estas herramientas lo verifican en cada envío al repositorio, y el envío se rechaza si alguna falla:

Herramienta	Ámbito	Qué verifica
`ruff check`	Backend	Estilo PEP 8, nombres, importaciones ordenadas, seguridad, simplificaciones
`ruff format`	Backend	Formato uniforme: no se discute dónde va un salto de línea
`mypy`	Backend	Tipos correctos en los 99 archivos
`pytest`	Backend	453 pruebas
`eslint`	Web	Reglas de React y TypeScript
`tsc --noEmit`	Web	Tipos en modo estricto
`vitest`	Web	41 pruebas

10.1 Reglas activadas en el analizador

La configuración vive en `backend/pyproject.toml`, en un solo lugar:

```
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = [
    "E",    # pycodestyle
    "W",    # warnings
    "F",    # pyflakes
    "I",    # isort — importaciones ordenadas
    "N",    # pep8-naming — nombres
    "UP",   # pyupgrade
    "B",    # bugbear — errores frecuentes
    "C4",   # comprehensions
    "SIM",  # simplify
    "S",    # bandit — seguridad
    "T20",  # sin print()
]
```

Dos supresiones están declaradas con su motivo, porque una excepción sin explicación es indistinguible de un descuido: `B008`, porque `Depends()` en argumentos por defecto es el patrón propio de FastAPI, y `N811`, porque importar `UUID as PgUUID` es el modismo estándar para no chocar con `uuid.UUID`.

10.2 Estado actual

```
$ ruff check app tests
All checks passed!

$ ruff format --check app tests
126 files already formatted

$ mypy app
Success: no issues found in 99 source files

$ npx tsc --noEmit
(sin salida: sin errores)
```

11. Conclusión

El estándar de MONK GRAIN adopta las convenciones propias de cada tecnología —PEP 8 en Python, las de la comunidad de React en TypeScript— en lugar de inventar reglas nuevas, y añade tres acuerdos propios del proyecto: código en inglés y comentarios en español, comentarios que explican el porqué y no el qué, y trazabilidad explícita a la historia de usuario cuando una regla proviene de un criterio de aceptación.

Lo que convierte este documento en un estándar y no en una intención es la última sección: seis herramientas verifican cada regla en cada envío, y hoy todas pasan sin hallazgos.